Research Article

INTERNATIONAL JOURNAL OF ADVANCED ROBOTIC SYSTEMS

International Journal of Advanced Robotic Systems
May-June 2019: 1–14

DOI: 10.1177/1729881419853182
journals.sagepub.com/home/arx
SAGE

Finite state automaton based control system for walking machines

Razeen Hussain[1], Teresa Zielinska[1] and Rene Hexel[2]

Abstract

Walking machines have proved to be an important invention as they do not require any prepared surface, making them ideal for applications involving unexplored environments. They are equipped with a large number of actuators and sensors to achieve a robust locomotion; thus, a systematic approach to designing their control system is required. This article presents a functional control structure based on the logic-labelled finite state automaton approach developed for walking machines. A general control structure is presented and a hexapod robot is used to verify the practicability of the design.

Keywords

Walking machines, control system, real-time control, finite state machines, behaviour models

Date received: 13 June 2018; accepted: 3 May 2019

Topic: Robotics Software Design and Engineering
Topic Editor: Anis Koubaa
Associate Editor: Mohamed Khalgui

Introduction

One of the most important outcomes of biological evolution is the ability to walk since it does not require any prepared surface. Traditional mobile robots mostly use wheels which work well with only flat surfaces. For robots to be able to navigate in unknown environments, it is imperative for them to imitate the walking behaviour of biological species. Consequently, robotics has evolved to address this issue by coming up with the walking machines. The multi-legged walking machines do not require all appendages to be touching the ground making them more suitable for utilisation in uneven environments.

Many types of walking machines have been developed. Bipedal robots imitate human beings, however, for a legged robot to be statically stable, it should comprise three or more legs. For this reason, quadrupeds and hexapods are the most common walking machines. These robots have a large number of actuators and many sensors for their locomotion. It is evident due to their complex nature that a systematic approach for designing the control system of such machines is crucial for its closed-loop performance.

Mechanical design and motion generation for the multi-legged walking machines have been the main focus of researchers.[1–4] Although these are important aspects for the development of robots, it should be noted that the control system realisation problems have been a hurdle in their successful practical implementations.

By studying insect locomotion, it can be seen that they have different gaits and switch between them based on feedback coming from sensory neurons. Similar behaviour can be associated with walking machines. The robot should be able to alter its gait based on sensory feedback, adapting to the changing terrain. The gaits can be interpreted as

[1] Faculty of Power and Aeronautical Engineering, Warsaw University of Technology, Warsaw, Poland
[2] School of Information and Communication Technology, Griffith University, Brisbane, Australia

Corresponding author:
Razeen Hussain, Faculty of Power and Aeronautical Engineering, Warsaw University of Technology, ul. Nowowiejska 24, Warsaw 00-665, Poland.
Email: razeenhussain@hotmail.com

different states of the robot locomotion, making it ideal for representation as a finite state machine (FSM).

The purpose of this research is to elaborate and test the functional structure of a control system using the logic-labelled finite state machines (LLFSMs) approach. The functional structure under consideration is being designed for a walking machine performing exploration tasks. The main focus is on using a systematic approach to create a system having real-time capabilities.

This article is structured as follows: the state of the art related to control system design is presented first with focus on walking machines, followed by motivation of the work done and its context. The overall developed control structure is discussed next. The next sections describe the various FSMs developed, namely the global navigation and the local navigation FSMs along with the data repositories illustrating the whiteboard concept. Next, we demonstrate the utility of the concept through a real-time implementation using dedicated simulation tools. We illustrate the essential cases and explain the corresponding actions of the designed controller. The final section emphasizes key findings with concluding remarks.

State of the art

The control system of any robot is a crucial component for its success. It dictates how the robot behaves. Usually robots exhibit reactive behaviour. Thus, sensors play an important role for optimal closed-loop control.

Typically, there are many sensors present in the system, acting as the input for the control system. A global positioning system (GPS) allows the evaluation of the starting position of the walking machine, whereas a compass gives the heading direction (yaw angle) towards the goal. For detection of terrain conditions, the walking machine is equipped with inclinometers which calculate the roll-and-pitch angles. Proximity sensors allow detection of obstacles, digital encoders give feedback on the position of the leg joints, while the leg ends are equipped with contact or force sensors as well. The terrain properties govern the gait of the walking machine. The control system must be capable of handling all the gaits and the sequence of the leg transfer motion from one periodic gait to another.

One of the most essential principles in designing a modern software architecture is modularity. The main system is divided into several smaller modules, each responsible for a specific task, thus, allowing for a system developed for one robot to be used for another simply by replacing a module. The arrangement of these modules along with how they interact determines the control structure of the robot. Although there are many methods to arrange these modules such as the subsumption architecture,[5,6] the hierarchical approach[7–14] remains the most popular for walking machines.

Real-time systems need to react to external events within specific time constraints. Thus, the correctness of such systems depends not only on producing the adequate response but also on temporal factors, that is, when the response is produced. The real-time operating system (RTOS)[15] is usually incorporated into the embedded system with regard to robotic applications. Over the years, many RTOS have been developed such as RTAI,[16] QNX,[17] and VxWorks.[16] QNX is considered the best RTOS for complex walking machines' control systems as it offers fast and predictable performance and has an excellent architecture for implementing a robust distributed system.

Although many robotic control systems[18–21] have been developed with real-time capabilities for walking machines, the systematic FSM methods are not yet applied. FSM, a tool in model-driven engineering, is a computational model based on a system a finite number of states. An FSM is represented as a directed arc with nodes representing the system states and the arrows representing the state transitions. At any given time, only one state can be active. The machine can transition to another state based on logical statements that need to be proved by an inference engine. The LLFSM approach offers a reliable, robust and quicker design solution. It is a promising recent tool that has demonstrated to be an effective technology for modelling robotic control systems as demonstrated by Estivill-Castro et al.[22] and Figat et al.[23]

Motivation

The presented work is motivated by studies done by Estivill-Castro and Hexel for designing fast reactive systems[24] in which the FSM approach was successfully applied to timing-critical systems and by Zielinska and Heng on the development of real-time control systems for quadrupeds and a team of autonomous hexapods.[14,19]

In the latter example, the authors utilized the concept of multi-processing, using semaphores for synchronisation, and the QNX RTOS (C language) 'send', 'receive' and 'reply' commands. When sending the message ('send' command), a response was always required, resulting in a synchronous, sequential realisation of process actions (reflected in the sequence of the program commands), that is, after dispatching the 'send' command, the sending process is suspended. The associated message data reach the receiving process (through its 'receive' command), which then responds with a 'reply' command (potentially containing response data resulting from the receiver's processing between 'receive' and 'reply'). Only once the 'reply' command is received, will the sender process continue. Analogically, the receiving process will be suspended on the 'receive' command until message/data are provided by the sender through its 'send' command. This combination of blocking 'send' and 'receive' operations is often referred to as a rendezvous and ensures tight synchronisation between sender and receiver.

With this mechanism, timely data transfer without suspending the global navigation and local navigation processes was possible but with an intermediary process

acting as a data deposit box. However, this approach does not scale to complex systems, and even for simple architectures, the applied communication concept needs careful data handling and messaging[19] with clear indication which process is the receiver and which one is the sender. Of course, the senders to some processes can be receivers from another ones. The tested system worked well allowing to implement complex behaviours – it was easy to add or replace the walking gaits or to modify the navigation strategy, however, the communication channels (send, receive, and reply) between processes limit the possibility of global rearrangement of process communication. To overcome the limits and complexities of such data-driven coordination of processes, we decided to adapt the FSM-based approach that does not have this limitation and has successfully been tested in the software systems in the first example.

Acknowledging the system-level impact of communication bounds,[19] we apply the whiteboard concept that defines the semantics of communication only in terms of data. With this approach, 'everybody' can access any data in the whiteboard repository and (theoretically) at any time. However, to avoid any complexity resulting from explicit synchronisation, we need implicit synchronisation for the proper coordination of system actions which is provided by the deterministic timing of a scheduled arrangement of FSMs.[24] The system actions previously implemented and tested using the data-driven approach were transferred to FSM structures and implemented using the MiEditLLFSM modelling tool.

Our main contribution is the walking machines' scalable control system design concept using a deterministically scheduled FSM concept (equivalent in structural composability to a multi-threaded architecture, but akin in complexity to a single-threaded system) with whiteboard communicated data repositories. The concept is free of communication bounds, however, all the state realisations must be carefully designed. The present article delivers the general concept of control system development focusing on walking machines. For the demonstration of the system work, the navigation task with obstacle avoidance was implemented. That choice allows us to compare the system behaviour with previously developed control and navigation system for the team of walking machines.

Control system structure

The general decomposition of the control system and the tasks assignment was proposed considering our previous experience with the design of QNX-based motion-control system for a team of autonomous walking machines. The concept was proved as being universal and efficient in the tested complex real-time scenarios.[19,20] For implementing the system functions in this work, we decided to use the FSM approach instead of event-driven design applied previously. The main advantage of using FSM is seen in its universality (application is not limited to RTOS) and efficiency in system prototyping as it is illustrated.

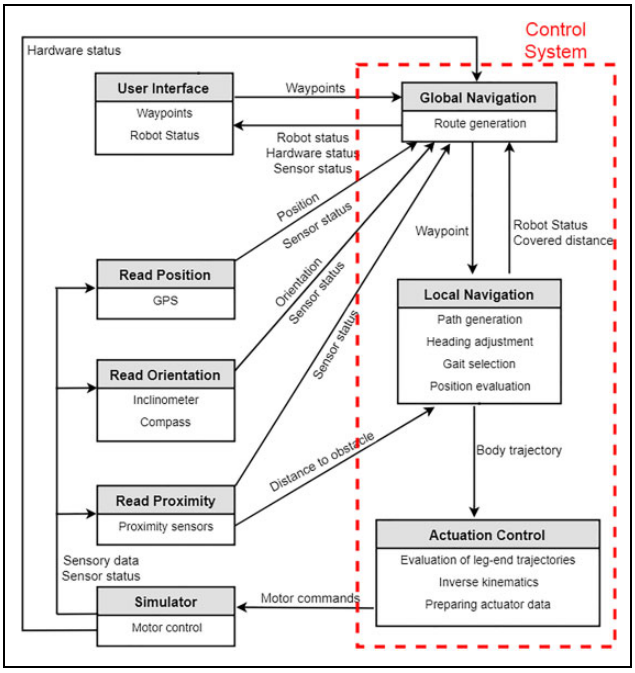

Figure 1. Detailed overview of the subsystems.

The developed control structure divides the overall system into smaller subsystems which are arranged in a hierarchical (top-down) structure. Such a hierarchical control scheme that was chosen as the developed control structure is not for a specific walking machine but can be used with any walking machine, regardless of the number of actuators and sensors attached to it. A subsystem can be replaced by another one as suited for the walking machine. This incorporates modularity into the system. The modular flexibility of the control structure allows it to be easily distributed over a network of microprocessors.

The general communication method is such: the highest level process sends the task demand to the level below (middle level), which in turn sends its demand to the level below it. All processes are self-reliant and equipped with tools to meet the demand. However, only when the demand cannot be met, the upper subsystem (the level directly above the level which is not able to fulfil the demand) is informed and a new demand is requested. Figure 1 illustrates the detailed breakdown of the control system processes. The main tasks associated with each process are also mentioned. Each process in the control structure is treated as a separate finite state automaton.

Since the activities of the sensory and user interface subsystems are simple in comparison with the global and local navigation subsystems, for brevity only the latter is discussed here. Inverse kinematics of walking machines is a widely researched area, however, it is specific to each walking machine. Therefore, the actuation control subsystem responsible for the inverse kinematics is also not discussed here.

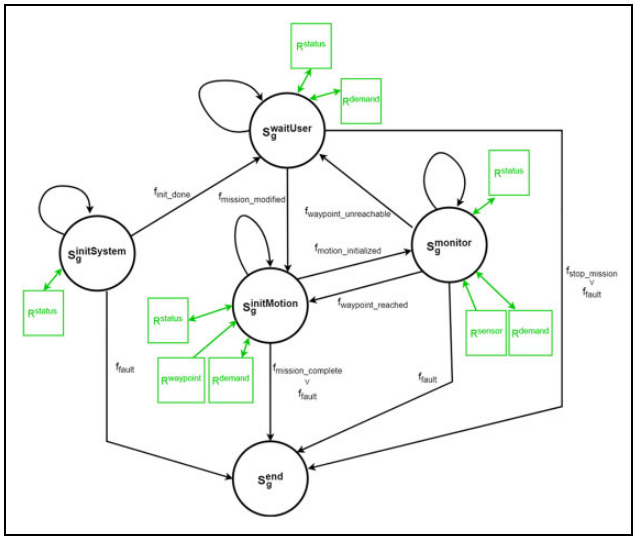

Figure 2. Activities of the global navigation subsystem represented by an FSM. The circles are marking the states, the branches are marking the events, and the green boxes are marking the accessed/deposited data in the whiteboard. The state transitions are triggered by data taken from the whiteboard. For whiteboard connection, the arrows are indicating the data flow (one way or two ways). FSM: finite state machine.

Global navigation FSM

The global navigation automaton is the highest level subsystem in the control structure and is responsible for route generation. It interacts with the user interface automaton, receiving target waypoints and requesting further instructions in case of a locked path. Moreover, the global navigation process monitors the mission, calculating the heading error and evaluating the position errors. This information is deposited on the status repository which is subsequently used by the local navigation subsystem. Figure 2 shows the activities carried out by the global navigation subsystem as described by an LLFSM.

Each state of the global navigation automaton is described below:

- $S_g^{\text{initSystem}}$: This is the initial state of the automaton. It sets up the data repositories and performs a check that the robot system is working properly.
- S_g^{waitUser}: When the automaton enters this state, requests are sent to the user. These requests include request for a new mission (if there is no active mission available) or a request to modify or stop the mission in case of a locked path.
- $S_g^{\text{initMotion}}$: This state reads from the waypoint repository and communicates the next goal position to the local navigation automaton and then transitions to the S_g^{monitor} state.
- S_g^{monitor}: While the local navigation automaton is navigating the robot through the unknown environment, this state prepares the basic status messages (position error, heading error, etc.) for the other processes. It uses data from the sensor repositories to calculate the various error in the system.
- S_g^{end}: In case of a system failure or a stop mission request from the user, the automaton transitions to this state which terminates the process.

Local navigation FSM

The local navigation subsystem is the middle-level functional process in the hierarchy. It is responsible for detailed path navigation and gait selection. It uses information from the proximity sensors to produce a collision free path for the robot, fulfilling the route demand received from the global navigation subsystem. The information from the inclinometers is used to define the type of gait motion.

The navigation algorithm defines the body trajectory. This demand is sent to the lowest level in the hierarchy, that is, actuation control subsystem which generates the motor commands. Based on the body trajectory, the control system should also be able to evaluate the new position of the robot on its own since the GPS data are unreliable. Figure 3 shows the activities carried out by the local navigation subsystem as described by an LLFSM.

The states of the local navigation automaton are:

- $S_l^{\text{loadTarget}}$: This is the initial state of the automaton. It sends a request to the global navigation automaton that the current goal position has been reached and therefore requests a new target point. When a target waypoint has been loaded, it transitions to the S_l^{moveFwd} state.
- S_l^{moveFwd}: This state generates the body trajectory for a straight line motion. It also keeps track of obstacles present in the straight line motion by reading the sensor repository and transitions to the S_l^{moveLeft} state in case an obstacle is detected in the forward direction. The heading errors deposited in the status repository by the global navigation automaton are used to perform heading corrections when necessary.
- S_l^{moveLeft}: The gait motion for leftward motion is generated in this state. When the forward path clears, it transitions back to the S_l^{moveFwd} state. It also keeps track of the robot position within the corridor using a step counter to ensure the robot navigates in its defined proximities.
- $S_l^{\text{moveRight}}$: When there is no path around the obstacle from the left side, the automaton transitions to this state which generates the rightward gait motion.
- S_l^{turnLeft}: Whenever there is a requirement to perform heading correction with a left turning motion, the automaton transitions to this state.
- $S_l^{\text{turnRight}}$: Similarly to the S_l^{turnLeft} state, when a heading correction is required (but with a right turn motion), the automaton transitions to this state.

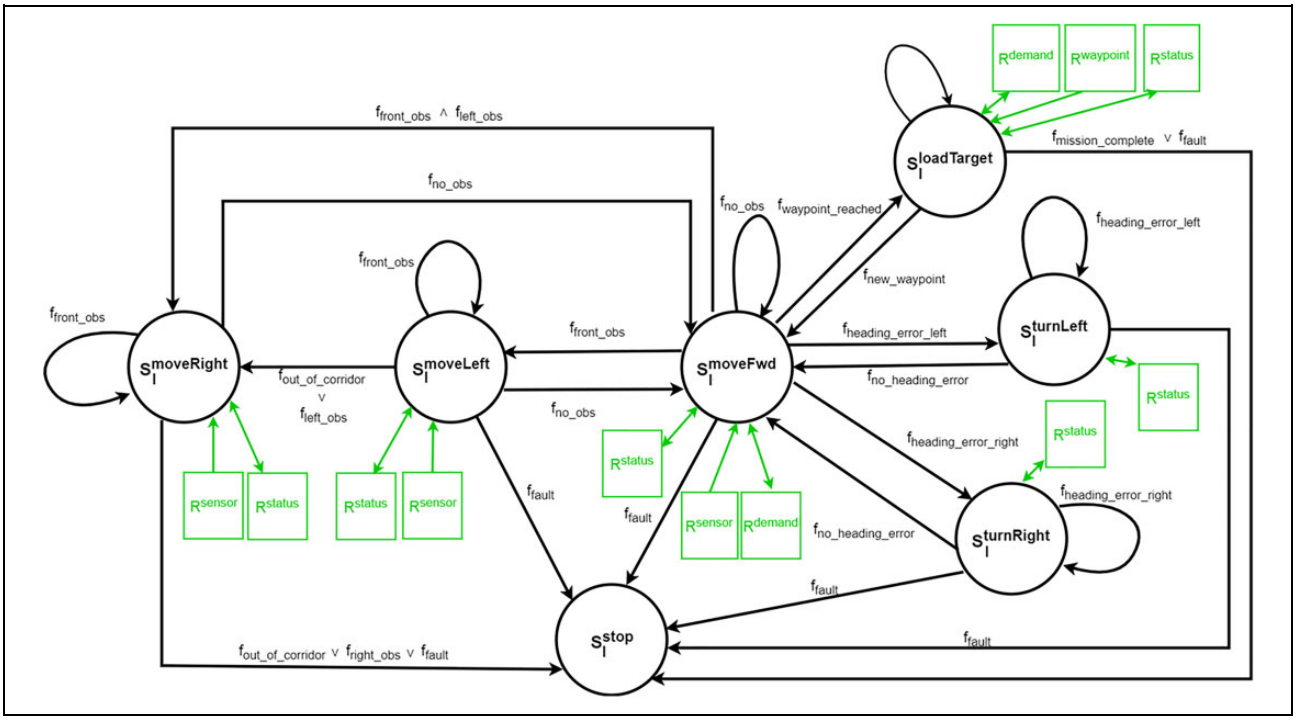

Figure 3. Activities of the local navigation subsystem represented by an FSM. The state transitions triggered by sensed conditions are adequately labelled, for example, $f_{\text{front\_obs}}$, $f_{\text{heading\_error\_left}}$. FSM: finite state machine.

- S_l^{stop}: In case of a system fault or the mission has been completed, the motion of the robot needs to be stopped. This state ensures the robot has stopped and returns the robot to its initial configuration.

The walking machine's locomotion is characterized by a series of leg displacements known as a gait. There are many possible gaits, however, for the purpose of experimentation, only two periodic gaits are incorporated in the control system, namely the tripod gait and the wave gait. The local navigation subsystem is also responsible for the selection of these gaits. A two-tier definition of the behaviours is used. The upper level (the local navigation FSM as shown in Figure 3) is an FSM which switches between one and more of these behaviours. The lower layer governs which type of gait to use. This two-tier description follows the concept of hierarchical FSMs. The generic template of the sub-behavioural FSM modelling the behaviours of the local navigation subsystem is demonstrated in Figure 4. This arrangement of hierarchical FSMs is used with the S_l^{moveFwd}, S_l^{moveLeft} and $S_l^{\text{moveRight}}$ behaviours.

The various states of the sub-behavioural FSM for each motion state of the local navigation subsystem are described below:

- S_s^{tripod}: This state generates the tripod gait. The FSM transitions to this state when the motion surface is relatively flat.
- S_s^{wave}: The wave gait is implemented in this state. Since this is the most stable gait, the FSM transitions to this state when a surface inclination is detected.

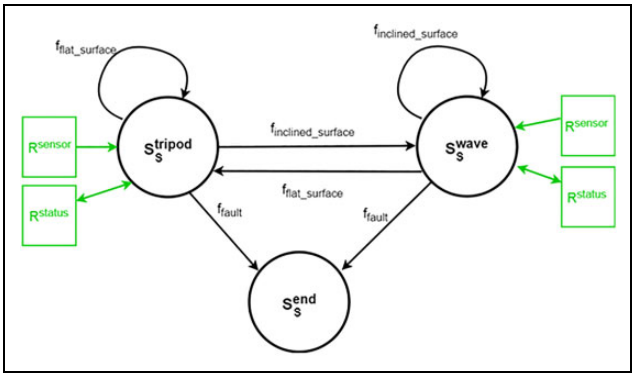

Figure 4. Activities of the motion behaviours represented by an FSM. FSM: finite state machine.

- S_s^{end}: In case of a fault, the sub-behavioural FSMs transition to this state, subsequently terminating the process.

State activities

The state activities represent two levels of abstraction:

1. The states providing and serving the walking machine motion.
2. The machine representing the overall system states.

The states denoted with S_l^a, S_s^a belong to the first level of abstraction, while the states denoted with S_g^a belong to the

second level. The states $S_1^{\text{moveRight}}$, S_1^{moveLeft} and S_1^{moveFwd}, producing the body locomotion, are realized by wave (S_s^{wave}) or tripod (S_s^{tripod}) gait depending on the terrain. Body turning is achieved by displacing the legs in such a way that the body is rotated by a sufficient angle. The gait generation problem (the gait diagrams and generation of leg-end trajectories assuring the realisation of body trajectory) is well-studied and is out of scope of this work. Interested readers can find the summaries on gaits generation and implementation with the experimentally tested self-localisation method in the literature.[14,19,25,26]

The detailed realisation of actions serving the system depends on the physical system, for example, on the processor architecture, available indicators of hardware components status, on the level and availability of fault information and the like. Hardware and software dependent examples are shown by Zielinska and Heng[19] for another concept of control system development. In simulations, the system serving actions are naturally much simpler. Suffice it that here we summarized the global and local navigation FSM state activities.

OnEntry and OnExit conditions (denoted by f_c) are produced within the states using the whiteboard readings (represented by green boxes). For example in Figure 3, state S_1^{moveLeft} reads the status of the system (R^{status}) and the sensory data (R^{sensor}) continuously from the whiteboard. If the status is different from normal, then the f_{fault} as the OnExit condition becomes valid and the state transfers to S_1^{stop}. When the sensory reading (R^{status}) indicates the obstacle on the left, $f_{\text{left\_obs}}$ is set as valid by S_1^{moveLeft}. The data from another sensor (GPS) combined with local odometry are used for checking whether the machines are out of acceptable corridor limits. If this happens, then the $f_{\text{out\_of\_corridor}}$ condition becomes valid. For S_1^{moveLeft}, $f_{\text{left\_obs}}$ and $f_{\text{out\_of\_corridor}}$ are the two of four possible OnExit conditions. If at least one of $f_{\text{left\_obs}}$ and $f_{\text{out\_of\_corridor}}$ is valid, the state S_1^{moveLeft} transfers to $S_1^{\text{moveRight}}$ (by OnEntry $f_{\text{out\_of\_corridor}}$ or $f_{\text{left\_obs}}$) producing the side walk to the right side.

Data repositories

The interprocess communication is done through a central database holding all shared data. There are several data repositories represented in this central database, namely the sensor repository (R^{sensor}), the waypoint repository (R^{waypoint}), the demand repository (R^{demand}) and the status repository (R^{status}). These repositories are represented with green boxes in Figures 2to 4. The sensor repository holds information provided by various sensors attached to the walking machine. Table 1 describes the data represented in the sensor repository. The waypoint repository (see Table 2 for details) holds the set of goal positions for the robot as specified by the user. User requests such as demand for an update of robot location and modification of mission are represented in the demand repository as described in Table 3. The status repository (see Table 4 for details) contains information on various parameters that govern the navigation of the robot.

Table 1. Description of data contained in the sensor repository (R^{sensor}).

Data name	Data type	Description
Proximity sensors	Boolean array	Data from the proximity sensors. '0' and '1' indicate the absence and presence of obstacles, respectively. Each element of the array represents a proximity sensor. Default array size is 6.
Position sensor	Float array	Positional data from the geolocation sensors. Default array size is 2 representing the *x* and *y* coordinates.
Compass sensor	Integer	Data from the compass sensor, that is, yaw angle of the robot. Data are stored as degrees.
Inclinometer sensor	Integer array	Data from the inclinometers, that is, roll and pitch angles of the robot. Data are stored as degrees.

Table 2. Description of data contained in the waypoint repository (R^{waypoint}).

Data name	Data type	Description
Mission targets	Float array	Array containing goal positions within a mission. Default size of mission is 5 target positions.

Table 3. Description of data contained in the demand repository (R^{demand}).

Data name	Data type	Description
Demand for position	String	User demanding current robot location.
Demand for heading	String	User demands for current robot heading.
Demand to cancel mission	String	Cancel mission request from user.
Demand to modify mission	String	User requesting a modification in mission target points.

Implementation

The FSMs described above were implemented using the MiEditLLFSM modelling tool.[24] This tool was developed for the IT industry. Therefore, it can be used for modelling the activities of a robotic control system. The actions of the

Table 4. Description of data contained in the status repository (R^{status}).

Data name	Data type	Description
Evaluated position	Float array	Evaluated position using incremental localisation. Only *x* and *y* coordinates are evaluated.
Heading error	Integer	Calculated heading error.
Fault status	Boolean array	Status of faults that may occur during a mission. Three types of fault (sensor failure, embedded system fault and data corruption) are considered. '0' indicates no fault, while '1' indicates an occurrence of a fault.
Within corridor	Boolean	Robot is moving within the corridor limits.
Target reached	Boolean	Current target position has been reached.
Gait type	Integer	Type of gait currently used by robot. '1' and '2' indicates tripod and wave gaits, respectively.

individual states were defined using C++ code. When all the activities of the individual FSMs were specified, the executable C++ code was automatically generated by the tool. The execution of the FSMs was determined by a scheduler. Concurrency is maintained through a static schedule that guarantees atomicity by ensuring that only one step of the FSMs is executed in each time slot.

Selection of FSM implementation method

Event-driven approaches in which the transitions are labelled by events are more popular for modelling the actions of an FSM. Over the years, many frameworks[27–29] have been developed for the implementation of such models. However, the event-driven-based models require implementation as a set of concurrent threads which communicate with each other. This multi-threaded approach increases the load on the system as a supplementary process must manage the synchronisation of the threads and make sure there is no deadlock or thread starvation. Moreover, the idealised model of physical chance events, ordered on an infinitely dense time line can only be approximated in software, resulting in nondeterministic memory usage and timing, unsuitable for the requirements of reactive real-time systems.[30] A further drawback of the event-driven models is that the verification process may result in a combinational explosion as all combinations of the FSM states need to be correct. This considerably compromises the formal verification of the models both in the time domain and the value domain.

To resolve these issues, alternative approaches[24,31] have been proposed. Such approaches support single-threaded execution of multiple FSMs. MiEditLLFSM tool is one of the frameworks that use this approach. It follows the approach of Harel's statecharts[32] and defines the behaviours as a mathematical model.[33] In contrast to the asynchronous event-driven frameworks, this tool ensures synchronous FSMs. The MiEditLLFSM tool will be used to model the behaviour of the walking machines as it provides all the necessary tools for modelling.

MiEditLLFSM modelling tool

The MiEditLLFSM tool has been developed by MiPal. It represents an FSM as a graph, the states as nodes and transitions as arcs. The nodes are connected to each other by directed arcs. The arcs are labelled by Boolean expressions. This is in contrast to the event-driven approach where the arcs are labelled by events. The generality is not compromised as the Boolean expressions can also represent an event occurrence.

Each state of an FSM is attributed with three sections[34] which are described below:

- An OnEntry section contains the action that needs to be carried out when the system enters that state.
- An OnExit section contains the action that is executed when the system leaves the state and transition to another state.
- An Internal section contains the actions that are carried out when no transition is triggered. Unlike the other two sections which are executed only once, these actions are executed repeatedly until a transition condition becomes true.

Whenever a transition is enabled, the FSM switches to the next state and executes the OnEntry section of the new state. Each section only executes a single action. Multiple FSMs need to be executed simultaneously to ensure real-time capability in the system. The MiEditLLFSM scheduler ensures that at each time slice, one step of the FSM is executed satisfying concurrency in the system.

Communication via whiteboard

The whiteboard[35] is a structured global database which mimics the blackboard architecture.[36] It stores all the variables contained in the FSMs (data represented in the data repositories described previously) and allows access to all FSMs running on the system as represented in Figure 5. However, at any time only one FSM should have access to the whiteboard to provide lock-free atomicity. The FSM scheduler ensures this.

There can be three types of variables in the system:

- Local variables are available only within an FSM.
- Internal variables are shared among the various FSMs in the system.
- External variables are outside the system such as the data from the sensors.

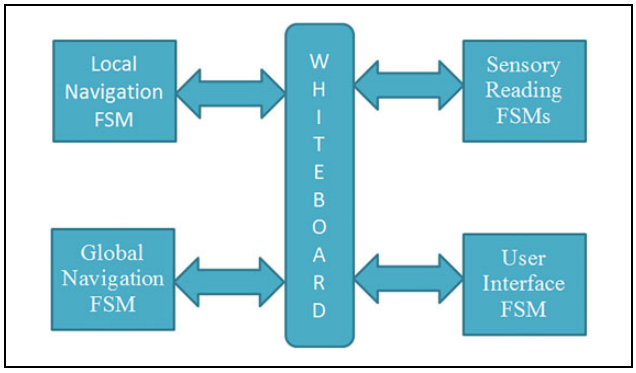

Figure 5. Communication between FSMs using the whiteboard. FSM: finite state machine.

Whenever a section of the current state executes, it first reads the whiteboard and makes local copies of the whiteboard variables. This is required so that all data being used by the action or transition function are recent.

Real-time implementation of concurrent FSMs

During the implementation, all three sections of the state were used. The OnEntry section associated with each state was used to setup the state variables. The OnExit section was used for the clean-up code. All the cyclic actions were implemented in the Internal section. Figure 6 shows a section of the implementation of the local navigation FSM described earlier. Only a particular section of the FSM (the section pertaining to S_1^{moveLeft}) is shown for illustration. The FSM can transition to S_1^{moveLeft} when the current state is S_1^{moveFwd}, and there is an obstacle present in its forward path (prox.obsFront is true). When the FSM enters S_1^{moveLeft}, its OnEntry section is executed first, that is, it resumes the activities of the submachine related to sideways walk towards left (referred to as BehaviourGaitLeft in Figure 6). The submachine is implemented according to Figure 4. After executing all the code in its OnEntry section, the Internal section is executed over a loop until one of the transition conditions becomes valid, for example, the path is blocked on the left side (prox.obsLeft is true). In that case, first the OnExit section of S_1^{moveLeft} is executed, that is, the activities of the submachine are suspended. After that the FSM transitions to $S_1^{\text{moveRight}}$. It should be noted that there are large amounts of codes in all OnEntry, OnExit and Internal sections of each state, so for brevity and illustration, only snippets of code are shown in Figure 6.

All FSMs described previously are executed simultaneously using a parallel programming structure. Each FSM is executed as a single-threaded process. The concurrent FSMs are arranged in a round-robin structure following the guidelines provided in the study by Estivill-Castro and Hexel.[24] This implies that each FSM gets the processor's resources for a ringlet (a pass over the cycle) execution. Once a ringlet completes its execution, the next FSM in the arrangement gets the processor's resources. This execution is governed by an execution token. When an FSM enters a suspended state, the execution token is passed to the next FSM in the arrangement.

During a ringlet, a local copy of the variables is made before execution of any section of the state. This ensures

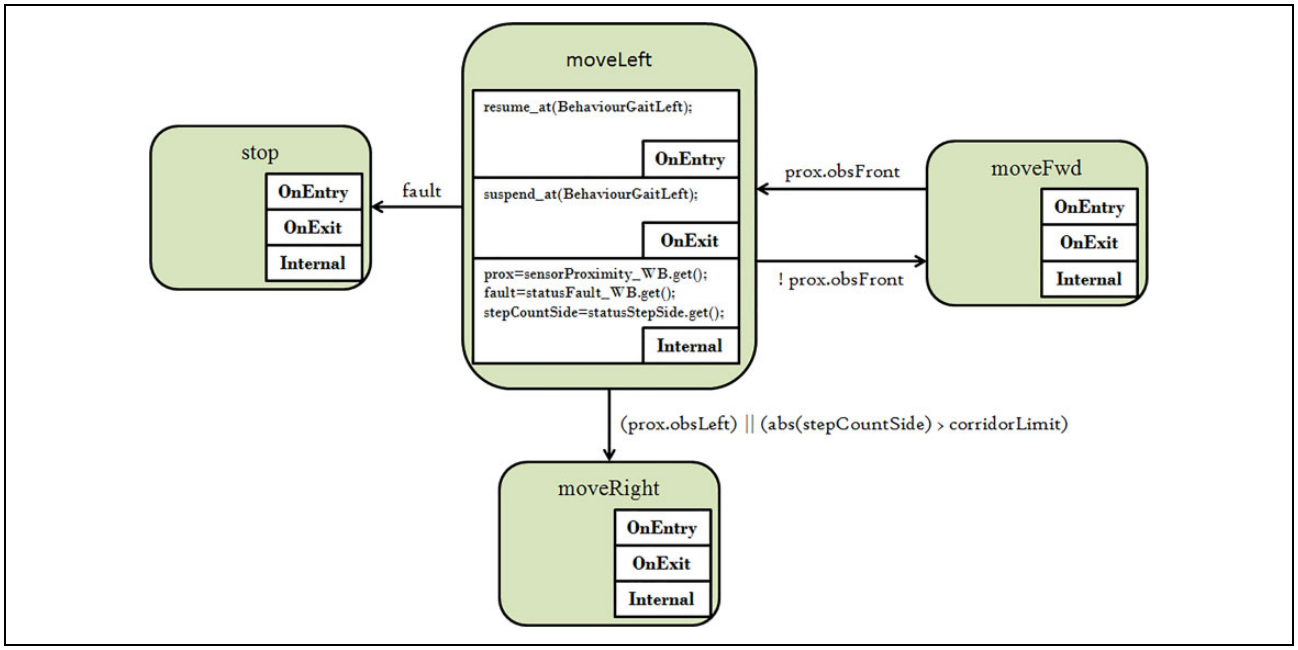

Figure 6. Implementation of local navigation FSM showing the OnEntry, OnExit and Internal sections of the states. Only the state transitions and code relating to state S_1^{moveLeft} are shown here. FSM: finite state machine.

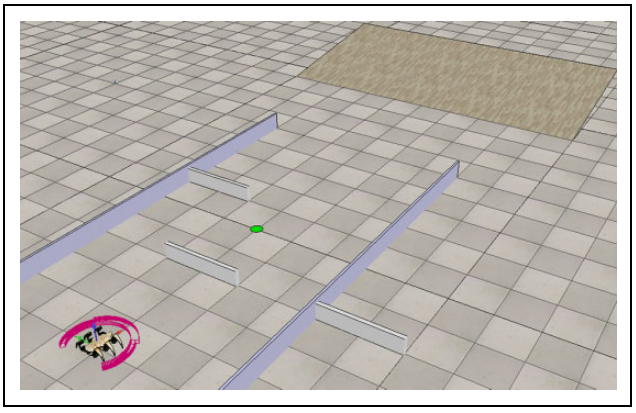

Figure 7. Simulation of the robot navigating in the unknown environment.

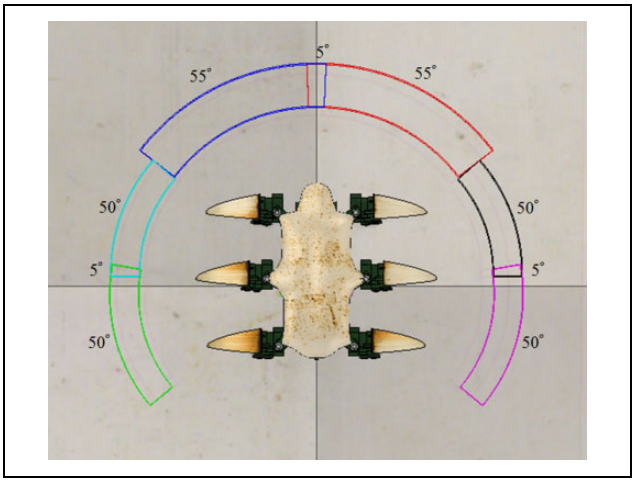

Figure 8. Arrangement and field of view of the proximity sensors.

that the data within an FSM are current and that no other concurrent FSM modifies the data during the state action execution, barring the need for semaphores and mutexes. This avoids concurrency issues and results in predictable timing required for real-time operation.

Simulation experiments

To verify the functionality of the proposed robot control system, the logic-labelled finite-state automaton-based control system was simulated using the Virtual Robot Experimentation Platform (V-REP) simulator.[37] V-REP is a cross-platform simulator allowing the creation of scalable, portable and easy to maintain models and scenes. Four different physics simulators (Bullet, Newton, ODE and Vortex) allow the simulation of soft and rigid body dynamics along with collision detection. Diverse selection of powerful and realistic sensors such as proximity and vision sensors is also available within the simulator. The ability to run scripts using a remote application programming interface is beneficial as it allows a convenient mechanism to control the simulation using the MiEditLLFSM tool.

The simulator is responsible not only for modelling the geometry of the environment and its dynamics but also responsible for modelling the activities of the robotic subsystems, that is, providing the data to the sensory modules, for example, the presence of obstacles in the robot's trajectory; reacting to commands received from the actuation control subsystem, for example, movement of leg-ends based on the trajectory received.

The simulation environment consisted of a set of obstacles of varying sizes randomly placed in an open space as represented in Figure 7. Inclined surfaces were also placed in the robot's path. It should be noted that the big walls seen on either side of the robot are not physical objects but rather define the corridor in which the robot must limit its motion. The corridor is determined by the start and goal positions and changes with each waypoint. A 3-m corridor width was chosen for the simulation experiments.

A built-in model of the ant hexapod robot was used. The ant has a rectangular body architecture and each leg has three degrees of freedom. GPS, compass and inclinometers were attached to the robot model to provide robot pose as feedback to the control system. Six proximity sensors were also attached to the hexapod with properly selected field of views as represented in Figure 8. Since the inverse kinematics of the leg trajectories is not the focus of this research, the IK mode provided with the built-in model was utilized for simplicity. Interested readers can visit the V-REP website[37] for details on the IK mode.

Various scenarios were simulated for the robot. The robot was made to navigate between a set of user-defined waypoints. The robot searches for a path between each waypoints. Ideally, the robot tries to follow the shortest path between the start and the goal positions, that is, straight line trajectory. When an obstacle blocks the desired path, the robot tries to navigate around the object. If a path exists around the obstacle, the robot continues its motion towards the target waypoint. However, in case no path exists within the robot workspace constraints, then a signal is generated and the situation is communicated to the user, while the robot awaits further instructions. This limited knowledge path search followed by the robot is similar to that of a bug algorithm.[38] Figure 9 represents examples of trajectories taken by the robot during the simulation experiments. The experiments were designed referring to the real-world scenarios that were tested previously using a different control system but based on the same navigation principles.[20]

Figure 9(a) shows the robot's obstacle avoidance behaviour. From the start position to point 1, the robot walks using the forward gait, being in state S_1^{moveFwd} (see Figure 3), as long as $f_{\text{no\_obs}}$ (no obstacle) is valid. At point 1, the sensor data report an obstacle and $f_{\text{front\_obs}}$ becomes valid

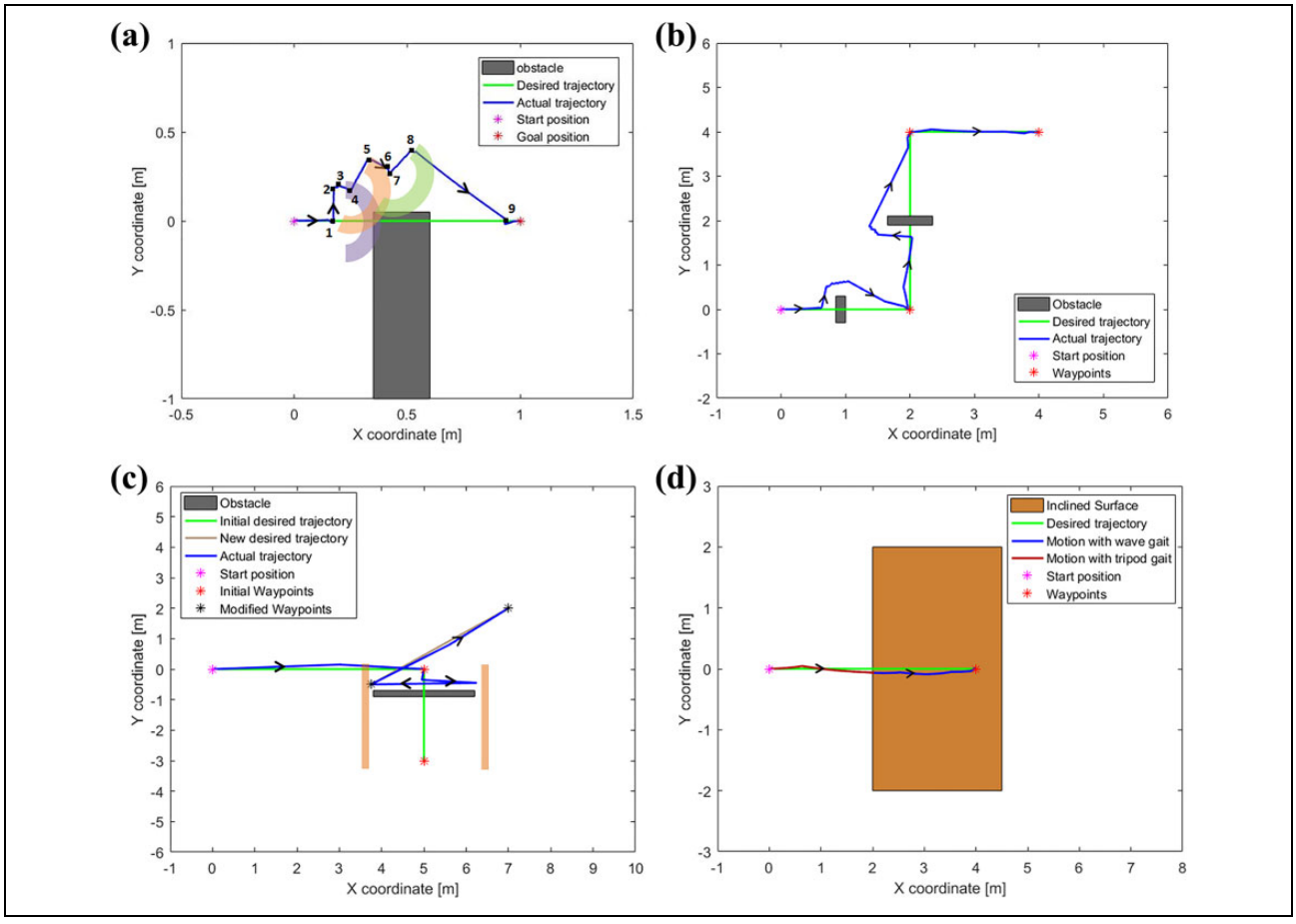

Figure 9. Robot trajectories in various tested scenarios. (a) Robot demonstrating obstacle avoidance. (b) Robot trajectory over multiple user-defined waypoints. (c) Path blocked behaviour. (d) Gait switch behaviour based on surface inclination

(purple arc marks the sensors range). The existence of obstacle $f_{\text{front\_obs}}$ results in a transition to state S_1^{moveLeft} (side gait to the left). It should be noted that in our system, we are biasing the obstacle avoidance to the left. When the front obstacle disappears (point 2), $f_{\text{no\_obs}}$ becomes valid, the FSM goes to state S_1^{moveFwd} (moving forward) but after a small distance (from points 2 to 3) the $f_{\text{heading\_error\_right}}$ is reported (point 3). As shown in Figure 3, the FSM transfers to $S_1^{\text{turnRight}}$, that is, the robot turns to the right until the heading error is cancelled. With valid $f_{\text{no\_heading\_error}}$, the FSM transfers to S_1^{moveFwd}, but immediately the front obstacle (orange arc) is detected $f_{\text{front\_obs}}$ (point 4). The situation is similar to the one at point 2, so the obstacle detection $f_{\text{front\_obs}}$ results in transition to S_1^{moveLeft} (side gait to the left) to the point 5. In point 5, the front obstacle disappears $f_{\text{no\_obs}}$, and the FSM goes to state S_1^{moveFwd} (moving forward) till it reaches point 6 where $f_{\text{heading\_error\_right}}$ is reported. Validity of $f_{\text{heading\_error\_right}}$ results in transition to $S_1^{\text{turnRight}}$ till the heading error is cancelled (point 7). Cancellation of a heading error ($f_{\text{no\_heading\_error}}$ is true) results in transfer to state S_1^{moveFwd}, but immediately the front obstacle (green arc) is detected $f_{\text{front\_obs}}$ (point 7). It results in transition to S_1^{moveLeft} (side gait to the left) till point 8 when the front obstacle disappears. Then with $f_{\text{no\_obs}}$ valid, the FSM transitions to S_1^{moveFwd}. The robot walks by forward gait till point 9 reaching the nominal path. Because the motion trajectory crosses the path to the target, the $f_{\text{heading\_error\_left}}$ becomes active which results in state transition to S_1^{turnLeft} till the heading error is cancelled. With $f_{\text{no\_heading\_error}}$, the S_1^{moveFwd} is activated and the robot reaches the goal position by forward walk.

A typical mission for the walking machine involves a set of target waypoints. Figure 9(b) illustrates one such mission. Three waypoints are sent by the remote user to the robot. The robot moves towards the first waypoint avoiding the obstacle present in its path. The state transitions are similar to the ones explained for Figure 9(a). Once the first waypoint is reached, $f_{\text{waypoint\_reached}}$ becomes valid and the FSM transitions to $S_1^{\text{loadTarget}}$. The local navigation layer informs the global navigation layer through the whiteboard and awaits the new target location. Meanwhile, the global navigation FSM (see Figure 2) transitions to $S_g^{\text{initMotion}}$ and

writes the new waypoint, sent earlier by the user at the start of the mission, on to the whiteboard, initialising the next motion. The $f_{\text{motion\_initialized}}$ becomes valid and the FSM transitions to $S_{\text{g}}^{\text{monitor}}$. The $f_{\text{new\_waypoint}}$ becomes valid triggering the transition to $S_{\text{l}}^{\text{moveFwd}}$. The motion continues until all target waypoints have been reached.

Similarly, Figure 9(c) shows the robot behaviour upon encountering a blocked path. The second waypoint in the mission is not reachable since it is blocked by an obstacle. At the point when the robot cannot find a path around the obstacle from either the left or right side, $f_{\text{waypoint\_unreachable}}$ becomes valid and the FSM transitions to $S_{\text{g}}^{\text{waitUser}}$ (see Figure 2). The user is notified about the blocked path and new waypoints are sent to the robot. Once the user initiates the modified mission, $f_{\text{mission\_modified}}$ becomes valid and transition to $S_{\text{g}}^{\text{initMotion}}$ is performed. The robot continues its motion based on the modified mission. In Figure 9(c), it appears that a path exists between the first and the second waypoints, however, it should be noted that the robot is not a point object and its geometry needs to be taken into account.

Figure 9(d) illustrates the transition from tripod to wave gait when the surface is at an inclination. The inclined surface is marked by the orange rectangle. The robot transfers the gait to wave gait when entering the inclination according to the state transition diagram in Figure 4. Small discrepancies between the desired and realised trajectory can be noticed but they are corrected before reaching the target point (see state $S_{\text{l}}^{\text{moveFwd}}$ in Figure 3 and $f_{\text{heading\_error\_left}}$ which results in $S_{\text{l}}^{\text{turnLeft}}$).

In ideal cases, a smooth trajectory is usually preferred. In contrast, the robot can be seen exhibiting a slightly erratic trajectory. This is due to the fact that a small tolerance of $\pm 5^{\circ}$ has been included in the robot heading direction error. Increasing the tolerance results in a smoother robot trajectory. However, increasing the tolerance too much causes a more erratic behaviour near the goal position. Thus, a compromise needs to be made.

A position error of ± 5 cm was observed for each waypoint during the simulation experiments. This error is due to the fact that the hexapod can only move at discrete distances corresponding to its stride length.

Comparative study

Zielinski et al.[39] offer deep insight to the methodology of control systems design with broad literature review. The authors claim 'Usually, architectures are presented by informal text descriptions supplemented by block diagrams, with varying level of details'; this makes the comparisons impossible. Very few publications[40–42] are tackling the problem of effective control systems for multi-legged walking machines; moreover, the descriptions are in general hardware and case oriented. On the other hand, due to the omnidirectionality, availability of many gaits and the need of locomotion adaptation to the terrain conditions, the control and navigation is rather complex.

When developing the architecture and working principles of the control system, several aspects are crucial. To this belongs:

- the robustness of the system to faults;
- scalability and composability;
- the system openness to modifications and additions;
- establishing correctness and efficiency;
- global accessibility of the most recent system data.

As already mentioned, the whiteboard concept allows to access a snapshot of the most recent data. The lack of communication bounds makes the system open to enhancements and modifications; moreover, even the previously neglected reactions to the system faults can be easily modelled and added. The correctness of the system must be guaranteed through an error-free implementation of subsystem states and interactions. The FSM-based approach has one more crucial advantage here; individual submachines as well as system actions can be easily tested by simulations and the same action structures can be used for developing the real system. This is a natural consequence of the independence of actions and data. The data can be delivered (and deposited) either by a simulator or by a real hardware. From the point of view of the system actions, this makes no difference, enabling testing of subsystems in isolation as well as in collaboration.

The disadvantage of the approach is the need for careful description of the activities (examples in Figures 2 and 3) without any mistakes in the definition of the accessed data and the state transitions. However, the danger of logic errors is rather the general drawback of developing complex control systems. As already mentioned, the simulation test is the remedy here. The tested part of the system can be directly moved to the real robot, and only the required hardware interfaces need to be added along with testing of system reaction to hardware faults.

Conclusion

A logic-labelled finite state automaton approach was presented to design a functional structure of a walking machine's control system. The control system was decomposed into three distinct functional subsystems arranged in a hierarchical structure, where each subsystem's activities were defined by an LLFSM.

The MiEditLLFSM tool,[24] an editor for LLFSMs, was used to model the behaviour of the FSMs. The tool allowed executable high-level behaviour models to be created facilitating both the modelling of the activities of the control system and the automatic code generation. The tool also manages the task scheduling for all the FSMs using a single sequential scheduler. Communication between the various FSMs was done using the whiteboard.

Generally, FSM-based approaches use an asynchronous event-driven multi-threaded model. However, such systems require a supplementary process to manage the synchronisation of the threads which increases the load on the processor. More importantly, multi-threaded, event-based models cannot guarantee predictability and are harder to verify. This makes them unsuitable for real-time robotic control systems. Thus, a single-threaded FSM execution approach was used to implement the FSMs.

The developed control system makes use of shared variables/memory. A problem with such an approach is that when one process is reading the shared memory, another process may alter the contents of that variable. This generally requires the use of mutexes and semaphores to protect the data. In contrast, a single sequential scheduler is used in the MiEditLLFSM tool which ensures that in a single time slice, only one FSM accesses the whiteboard/shared memory. Thus, there is no contention among the FSM processes during their execution.

The proposed control system allows for a remote operator to set waypoints for the walking machine. The path cannot be pre-planned and is generated in real time based on sensory information. The navigation algorithm implemented is a limited knowledge path planner.

The FSM-based approach was tested on a V-REP simulation which used a hexapod robot as an example. Various sensors needed for the control system to function correctly were incorporated in the physical structure of the hexapod. The robot was made to navigate in different environments. Obstacles were placed in the robots path to verify the obstacle avoidance algorithm.

As it was illustrated, the LLFSMs allowed to derive the architecture of complex control system in a clear and systematic way. Moreover, the system developed and tested in simulation can be easily applied in real-life applications, just by adding the hardware interfaces.

The work done provides a basis for designing robotic control systems based on the logic-labelled finite state automaton approach. As a next step, the implementation of the proposed approach will be extended to control an actual hexapod.

Nomenclature

Symbol	Description
S_d^a	S represents the state of the finite state machine (FSM), a is the name of the state and d represents the FSM (g for global navigation, l for local navigation and s for sub-behaviour).
R^b	R represents the data repositories in the whiteboard and b is the name of the data repository.
f_c	f represents the state transition condition and c is the name of the condition.
$S_g^{initSystem}$	Global navigation system initialisation state.
$S_g^{waitUser}$	Global navigation wait-for-user state.
$S_g^{initMotion}$	Global navigation motion initialisation state.
$S_g^{monitor}$	Global navigation monitor mission state.
S_g^{end}	Global navigation system termination state.
$S_l^{loadTarget}$	Local navigation load target state.
$S_l^{moveFwd}$	Local navigation forward motion state.
$S_l^{moveLeft}$	Local navigation leftward motion state.
$S_l^{moveRight}$	Local navigation rightward motion state.
$S_l^{turnLeft}$	Local navigation left turning motion state.
$S_l^{turnRight}$	Local navigation right turning motion state.
S_l^{stop}	Local navigation motion halt state.
S_s^{tripod}	Sub-behaviour tripod gait motion state.
S_s^{wave}	Sub-behaviour wave gait motion state.
S_s^{end}	Sub-behaviour process termination state.
R^{sensor}	Repository for sensory readings.
$R^{waypoint}$	Repository for mission goal positions.
R^{demand}	Repository for user demands.
R^{status}	Repository for system status information.
f_{init_done}	Condition guarding $S_g^{initSystem} \rightarrow S_g^{waitUser}$, signalling completion of system initialisation.
$f_{mission\_modified}$	Condition guarding $S_g^{waitUser} \rightarrow S_g^{initMotion}$, signalling the user has sent new or modified mission waypoints.
$f_{motion\_initialized}$	Condition guarding $S_g^{initMotion} \rightarrow S_g^{monitor}$, signalling the demand (waypoint) has been sent to and acknowledged by local navigation FSM.
$f_{mission\_complete}$	Condition guarding $S_g^{initMotion} \rightarrow S_g^{end}$ and $S_l^{loadTarget} \rightarrow S_l^{stop}$, signalling all the mission waypoints have been reached.
$f_{waypoint\_reached}$	Condition guarding $S_g^{monitor} \rightarrow S_g^{initMotion}$ and $S_l^{moveFwd} \rightarrow S_l^{loadTarget}$, signalling current waypoint has been reached.
$f_{stop\_mission}$	Condition guarding $S_g^{waitUser} \rightarrow S_g^{end}$, signalling user demand to stop/abandon the mission.
$f_{waypoint\_unreachable}$	Condition guarding $S_g^{monitor} \rightarrow S_g^{waitUser}$, signalling current waypoint cannot be reached.
f_{fault}	Condition guarding transitions to S_g^{end}, S_l^{stop} and S_s^{end}, signalling detection of a system fault.
$f_{new\_waypoint}$	Condition guarding $S_l^{loadTarget} \rightarrow S_l^{moveFwd}$, signalling demand (waypoint) has been received from the global navigation FSM.
f_{front_obs}	Condition in local navigation FSM, signalling presence of obstacle(s) in the forward path.
f_{left_obs}	Condition in local navigation FSM, signalling presence of obstacle(s) in the leftward path.
f_{right_obs}	Condition in local navigation FSM, signalling presence of obstacle(s) in the rightward path.
f_{no_obs}	Condition in local navigation FSM, signalling no obstacle in the forward path.
$f_{out\_of\_corridor}$	Condition in local navigation FSM, signalling breach of corridor limits.

Nomenclature. (continued)

Nomenclature. (continued)

Symbol	Description
$f_{\text{heading\_error\_left}}$	Condition in local navigation FSM, signalling heading error towards the left.
$f_{\text{heading\_error\_right}}$	Condition in local navigation FSM, signalling heading error towards the right.
$f_{\text{no\_heading\_error}}$	Condition in local navigation FSM, signalling heading error is within the tolerance.
$f_{\text{flat\_surface}}$	Condition in sub-behaviour FSM, signalling flat surface.
$f_{\text{inclined\_surface}}$	Condition in sub-behaviour FSM, signalling inclined surface.

Acknowledgements

The authors would like to thank the support of European Master in Advanced Robotics plus (EMARO+) and Pacific Atlantic Network for Technical Higher Education and Research (PANTHER) programs.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship and/or publication of this article.

Funding

The author(s) received no financial support for the research, authorship and/or publication of this article.

ORCID iD

Razeen Hussain https://orcid.org/0000-0002-7579-5069

References

1. Ding X, Wang Z, Rovetta A, et al. Locomotion analysis of hexapod robot. In: Miripour-Fard B (ed.) *Climbing and walking robots*. IntechOpen, 2010, pp. 291–309.
2. Tedeschi F and Carbone G. Design issues for hexapod walking robots. *Robotics* 2014; 3(2): 181–206.
3. Zielinska T, Goh T and Chong CK. Design of autonomous hexapod. In: *Proceedings of the first workshop on robot motion and control (RoMoCo)*, Kiekrz, Poland, 29 June 1999, pp. 65–69. New York: IEEE.
4. Saranli U, Buehler M and Koditschek DE. Design, modeling and preliminary control of a compliant hexapod robot. In: *Proceedings of 2000 IEEE international conference on robotics and automation* (ed BS Carlisle), San Francisco, CA, 24–28 April 2000, pp. 2589–2596. New York: IEEE.
5. Simpson J, Jacobsen CL and Jadud MC. Mobile Robot Control – The Subsumption Architecture and occam-pi. In: Welch P, Kerridge J and Barnes F (eds) *Communicating process architectures*. Amsterdam: IOS Press, 2006, pp. 225–236.
6. Li M, Yi X, Wang Y, et al. Subsumption model implemented on ROS for mobile robots. In: *2016 Annual IEEE systems conference (SysCon)* (ed B Rassa), Orlando, FL, 18–21 April 2016, pp. 1–6. New York: IEEE.
7. Pearson K. The control of walking. *Sci Am* 1976; 235(6): 72–86.
8. Minati L and Zorat A. A tree architecture with hierarchical data processing on a sensor-rich hexapod robot. *Adv Rob* 2002; 16(7): 595–608.
9. Odashima T, Luo Z and Hosoe S. Hierarchical control structure of a multilegged robot for environmental adaptive locomotion. *Artif Life Robot* 2002; 6: 44–51.
10. Brooks R. A robust layered control system for a mobile robot. *IEEE J Robot Autom* 1986; 2(1): 14–23.
11. Payton D. An architecture for reflexive autonomous vehicle control. In: *Proceedings of 1986 IEEE international conference on robotics and automation* (ed AT Bejczy), San Francisco, CA, 7–10 April 1986, pp. 1838–1845. New York: IEEE.
12. Brooks RA. A robot that walks; emergent behaviors from a carefully evolved network. In: *Proceedings of 1989 international conference on robotics and automation* (ed G Bekey), Scottsdale, AZ, 14–19 May 1989, pp. 253–262. New York: IEEE.
13. Cai AH, Fukuda T, Arai F, et al. Hierarchical control architecture for Cellular Robotic System – simulations and experiments. In: *Proceedings of 1995 IEEE international conference on robotics and automation* (ed F Harashima), Nagoya, Japan, 21–27 May 1995, pp. 1191–1196. New York: IEEE.
14. Zielinska T and Heng J. Development of walking machine: mechanical design and control problems. *Mechatronics* 2002; 12: 737–754.
15. Buttazzo GC. *Hard real-time computing systems: predictable scheduling algorithms and applications*, 3rd ed. New York: Springer, 2011.
16. Barbalace A, Luchetta A, Manduchi G, et al. Performance comparison of VxWorks, Linux, RTAI, and Xenomai in a hard real-time application. *IEEE Trans Nucl Sci* 2008; 55(1): 435–439.
17. Hildebrand D. An architectural overview of QNX. In: *Proceedings of the workshop on micro-kernels and other kernel architectures*, Seattle, WA, 27–28 April 1992, pp. 113–126. Berkeley, CA: USENIX Association.
18. Raibert M, Blankespoor K, Nelson G, et al. BigDog, the rough-terrain quadruped robot. In: *Proceedings of the 17th world congress* (eds MJ Chung, et al.), Seoul, Korea, 6–11 July 2008, pp. 10822–10825. New York: IFAC.
19. Zielinska T and Heng J. Real-time-based control system for a group of autonomous walking robots. *Adv Robot* 2006; 20(5): 543–561.
20. Zielinska T. Control and navigation aspects of a group of walking robots. *Robotica* 2005; 24(1): 23–29.
21. Simmons RG. Structured control for autonomous robots. *IEEE Trans Robot Autom* 1994; 10(1): 34–43.
22. Estivill-Castro V and Hexel R. Logic labelled finite-state machines and control/status pull technology for model-driven engineering of robotic behaviours. In: *26th International conference on software & systems engineering and*

their applications (eds A Canals and JC Rault), Paris, France, 27–29 May 2015. Paris: AFIS.
23. Figat M, Zielinski C and Hexel R. FSM based specification of robot control system activities. In: *11th International workshop on robot motion and control (RoMoCo)* (ed K Kozlowski), Wasowo, Poland, 3–5 July 2017, pp. 193–198. New York: IEEE.
24. Estivill-Castro V and Hexel R. Arrangements of finite-state machines – semantics, simulation and model checking. In: *Proceedings of the first international conference on model-driven engineering and software development (MODELSWARD)* (eds S Hammoudi, et al.), Barcelona, Spain, 19–21 February 2013, pp. 182–189. Setubal: INSTICC.
25. Zielinska T. Synthesis of control system: gait implementation problems. In: *Proceedings of the fourth international conference on climbing and walking robots* (ed K Berns), Karlsruhe, Germany, 24–26 September 2001, pp. 489–496. Bury St Edmunds: IEEE.
26. Zielinska T. Autonomous walking machines – discussion of the prototyping problems. *Bull Pol Acad Sci Tech Sci* 2010; 58(3): 443–451.
27. Mellor SJ and Balcer M. *Executable UML: a foundation for model-driven architecture*, 1st ed. Reading, MA: Addison-Wesley, 2002.
28. Wagner F, Schmuki R, Wagner T, et al. *Modeling software with finite state machines: a practical approach*. New York: CRC Press, 2006.
29. Harel D and Naamad A. The statemate semantics of statecharts. *ACM Trans Software Eng Method* 1996; 5(4): 293–333.
30. Kopetz H. Should responsive systems be event-triggered or time-triggered? *IEICE Trans Inf Syst* 1993; 76(11): 1325–1332.
31. Estivill-Castro V and Rosenblueth DA. Model checking of transition-labeled finite-state machines. *Commun Comput Inf Sci* 2011; 257: 61–73.
32. Harel D and Politi M. *Modeling reactive systems with statecharts: the statemate approach*. 1st ed. New York: McGraw-Hill, 1998.
33. Hopcroft JE, Motwani R and Ullman JD. *Introduction to automata theory, languages, and computation*, 3rd ed. Boston: Pearson, 2007.
34. Estivill-Castro V, Hexel R and Rosenblueth DA. Efficient model checking and FMEA analysis with deterministic scheduling of transition labeled finite-state machines. In: *Third world congress on software engineering* (eds X Huang, et al.), Wuhan, China, 6–8 November 2012, pp. 65–72. New York: IEEE.
35. Estivill-Castro V and Hexel R. Simple, not simplistic the middleware of behaviour models. In: *International conference on evaluation of novel approaches to software engineering (ENASE)* (ed L Maciaszek), Barcelona, Spain, 29–30 April 2015, pp. 189–196. Setubal: IEEE.
36. Hayes-Roth B. A blackboard architecture for control. *Artif Intell* 1985; 26(3): 251–321.
37. The robot simulator V-REP. http://www.coppeliarobotics.com/ (accessed 15 October 2017).
38. Ng J and Braunl T. Performance comparison of bug navigation algorithms. *J Intell Robot Syst* 2007; 50: 73–84.
39. Zielinski C, Figat M and Hexel R. Communication within multi-FSM based robotic systems. *J Intell Robot Syst* 2019; 93: 787–805.
40. Proetzsch M, Luksch T and Berns K. Development of complex robotic systems using the behavior-based control architecture iB2C. *Robot Auton Syst* 2010; 58(1): 46–67.
41. Chwila S, Zawiski R and Babiarz A. Developing and implementation of the walking robot control system. In: *Proceedings of the third international conference on man-machine interactions* (eds D Gruca, T Czachórski and S Kozielski),-Brenna, Poland, 22–25 October 2013, pp. 97–105. Cham: Springer.
42. Andreev A, Zhoga V, Serov V, et al. The control system of the eight-legged mobile walking robot. In: *Proceedings of the 11th joint conference knowledge-based software engineering* (eds A Kravets, M Shcherbakov, M Kultsova, et al.), Volgograd, Russia, 17–20 September 2014, pp. 383–392. Cham: Springer.